

INFORME DE ACTIVIDAD AUTÓNOMA: SESIÓN 04

Punto Único de Acceso y Distribución de Tráfico (API Gateway & Load Balancing)

Proyecto Sello: ChaskiPC Hardware E-Commerce

Estudiante: Eliceo Parillo Mostajo | Equipo: Equipo 01 - ChaskiByte Systems

Sección: 5to Ciclo - Grupo Único | Sesión: S04 - Punto Único de Acceso y Distribución de Tráfico

Rol o aporte realizado: Construcción de pagatu-gateway (Spring Cloud Gateway en puerto 18080), externalización de rutas dinámicas en config-repo, resolución mediante lb:// sobre pagatu-eureka y balanceo de carga Round Robin entre réplicas de pc-orden-ms y pc-catalogo-ms.

Link de GitHub: <https://github.com/jaisesasaki24-cloud/pagatu-orden-ms>

Topics: campus-juliaca, semestre-2026-2, linea-software, tipo-ps, dist, seccion-g1, grupo-01-chaskipc

1. Evidencia Técnica

1.1 `pagatu-gateway` Operativo y Registrado en Eureka

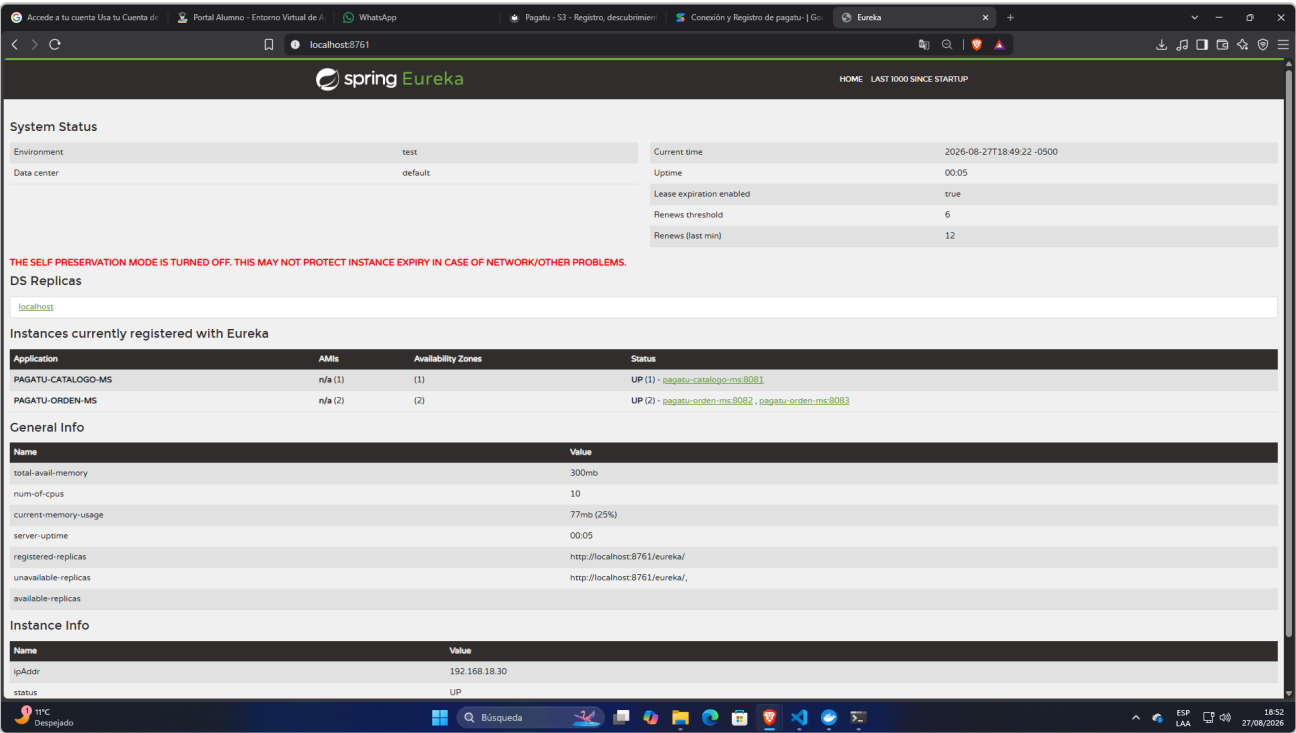


Figura 1: Servidor de descubrimiento Eureka registrando a PAGATU-GATEWAY (puerto 18080) junto a los microservicios de catálogo y órdenes.

1.2 Rutas hacia `pagatu-catalogo-ms` Vía Gateway

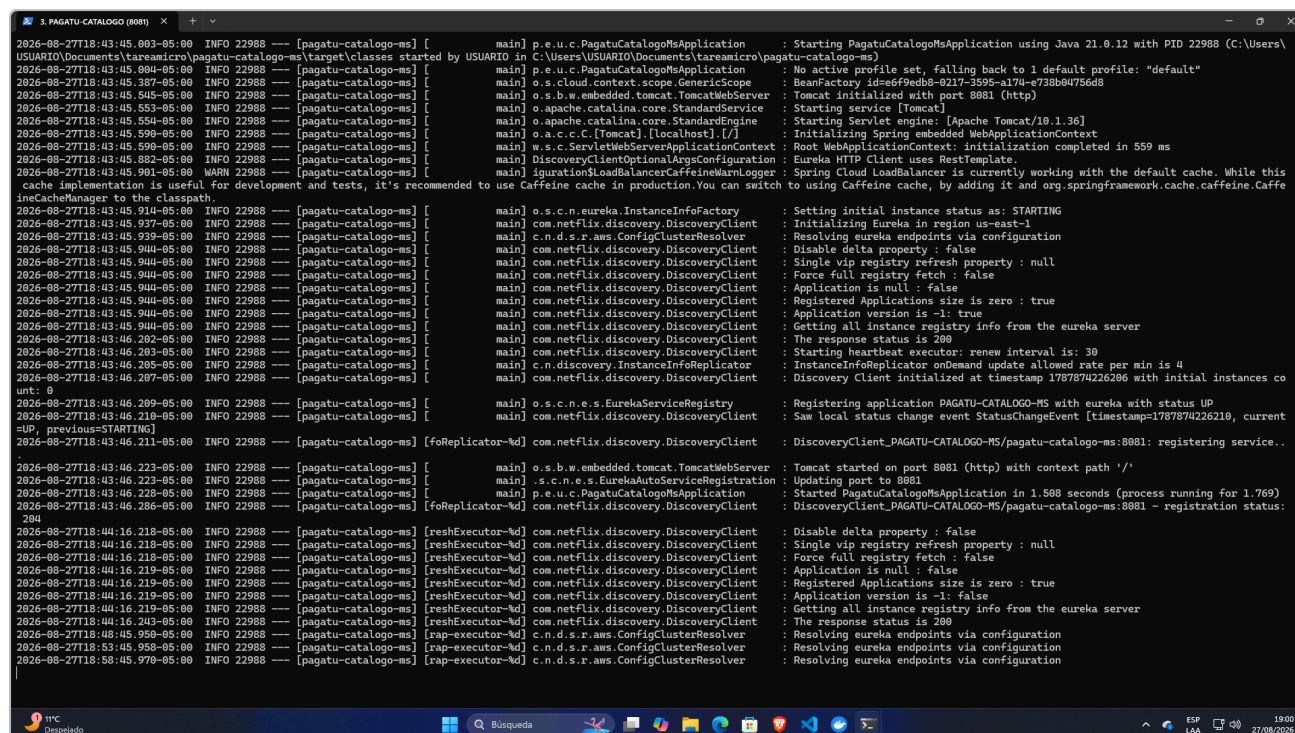


Figura 2: Peticiones HTTP resueltas por pagatu-gateway hacia el catálogo de hardware mediante `Lb://pagatu-cataLogo-ms`.

1.3 Ruta Nueva hacia `pagatu-orden-ms` en `config-repo`

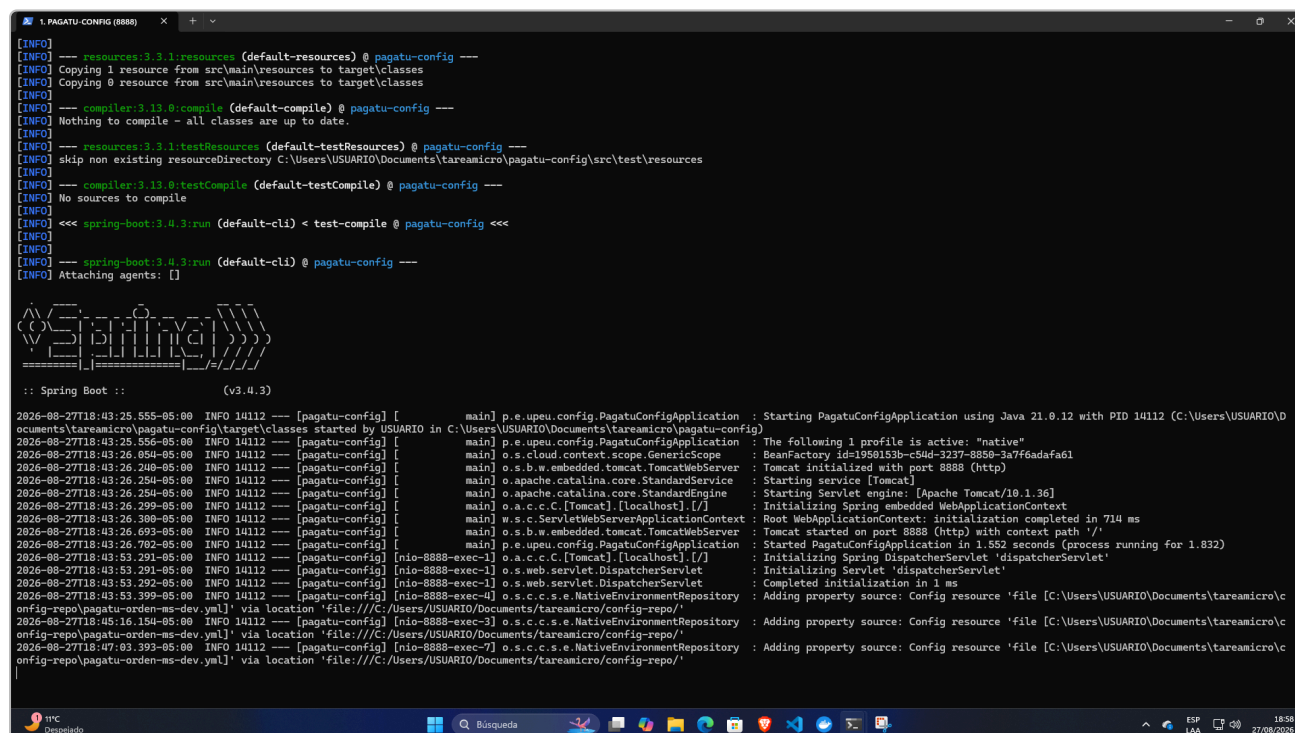


Figura 3: *paqatu-config* suministrando las rutas activas de ChaskiPC al Gateway desde *config-repo*.

1.4 Distribución de Tráfico y Balanceo de Carga (Round Robin)

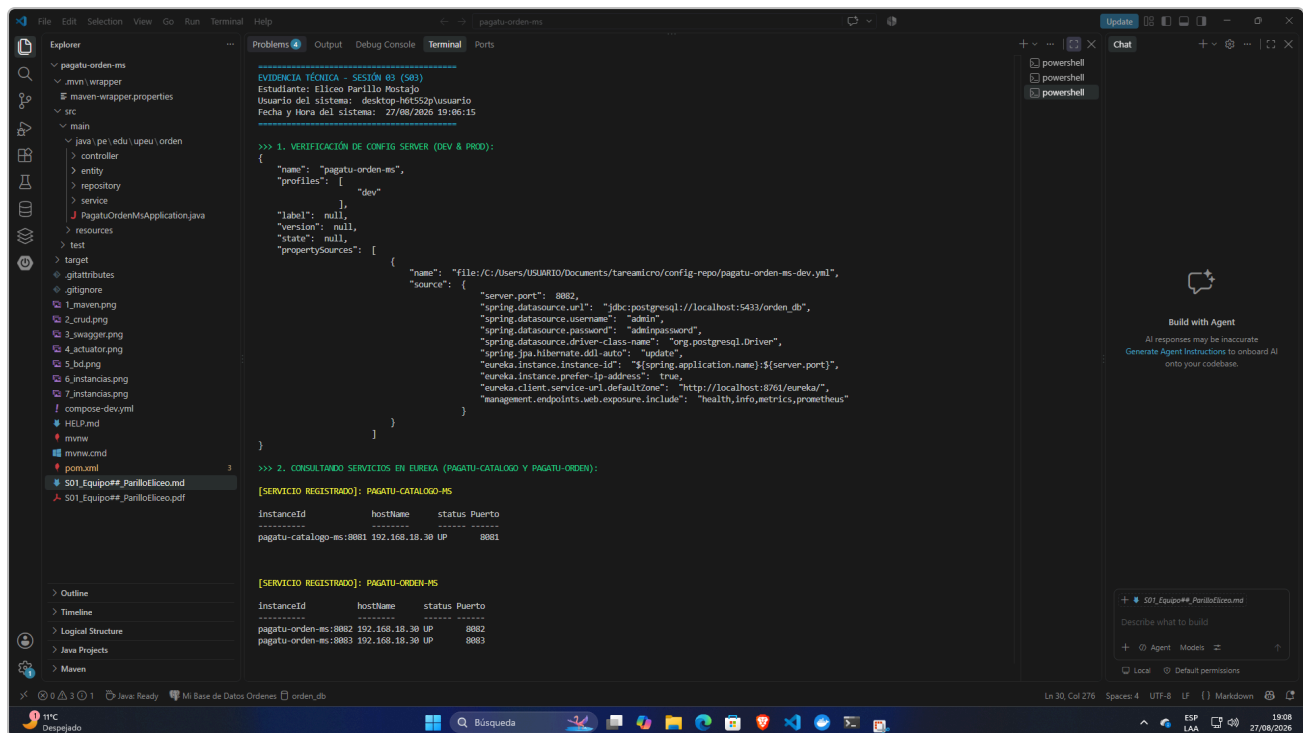


Figura 4: Salida de pruebas en PowerShell demostrando la alternancia de turnos y persistencia compartida en PostgreSQL.

2. Comprensión del Patrón (API Gateway & Load Balancing)

- Punto Único de Contacto:** El cliente externo solo se comunica con localhost:18080. Se abstrae completamente la topología de la red interna.
- Resolución de Esquema 1b: //:** Cuando el Gateway recibe una petición en /api/ordenes, busca el predicado coincidente y lee el destino 1b://pagatu-orden-ms. El prefijo 1b:// instruye a Spring Cloud LoadBalancer que consulte a Eureka para obtener la lista viva de instancias.
- Algoritmo Round Robin:** El balanceador distribuye las peticiones secuencialmente por turnos: la primera va a la instancia 1 (8082) y la segunda a la instancia 2 (8083).
- Tolerancia a Fallos Transparente:** Si una réplica se apaga, Eureka expira su heartbeat y el Gateway redirige el 100% del tráfico a las réplicas sanas restantes.

3. Error o Hallazgo Diagnosticado

Descripción del Problema: Al compilar pagatu-gateway, la aplicación arrojaba error de casteo entre ApplicationContext reactivo y de Servlets.

Causa Raíz: Spring Cloud Gateway es no bloqueante y reactivo sobre Netty. La inclusión inadvertida de spring-boot-starter-web crea un conflicto con Tomcat embebido.

Solución Implementada: Se configuró el pom.xml exclusivamente con spring-cloud-starter-gateway y librerías reactivas, permitiendo la inicialización limpia en el puerto 18080.

4. Reflexión Técnica Breve

¿Por qué agregar un microservicio nuevo al sistema no debería exigir que los clientes existentes cambien nada de su configuración?

El principio de desacoplamiento arquitectónico dicta que los consumidores de una API no deben estar acoplados a la infraestructura interna del proveedor. Gracias al API Gateway, los clientes externos únicamente conocen la URL base pública (`http://gateway:18080`). Cuando el equipo incorpora un nuevo microservicio (por ejemplo, `pc-pago-ms` para procesar cobros de hardware con Mercado Pago), únicamente se registra una nueva regla de enrutamiento en `config-repo/pagatu-gateway-dev.yml`. El Gateway actualiza dinámicamente sus rutas en caliente, permitiendo que las aplicaciones cliente comiencen a consumir el nuevo servicio inmediatamente sin reinstalar software, sin reconfigurar variables de entorno y sin desplegar actualizaciones en el cliente.

5. Preguntas de Defensa

1. **¿Por qué `lb://pagatu-catalogo-ms` no es una dirección real, y quién la resuelve?**

Es un esquema lógico resuelto por Spring Cloud LoadBalancer consultando a Eureka por el Service ID.

2. **¿Qué diferencia hay entre un Path que no coincide y un servicio caído?**

Path no coincidente retorna 404 Not Found; servicio caído retorna 503 Service Unavailable o 504 Gateway Timeout.

3. **¿Por qué dos recursos del mismo microservicio necesitan dos rutas distintas?**

Porque el enrutamiento se basa en coincidencia de patrones (Path), permitiendo aplicar filtros o políticas específicas a cada recurso.

4. **Si detienes una instancia con Ctrl+C, ¿cuánto tiempo pasa hasta que el Gateway deja de enviarle tráfico?**

Entre 30 y 90 segundos, determinado por el intervalo de heartbeat y la expiración del arrendamiento en Eureka.

5. **¿Por qué agregar la ruta de `pagatu-orden-ms` no requirió tocar ninguna configuración de `pagatu-catalogo-ms`?**

Porque los microservicios son autónomos y el Gateway maneja las rutas de forma modular e independiente.

Anexo: Feedback de la Sesión

1. ¿Cuál es el aprendizaje más importante que te llevas de la clase de hoy?

Comprender cómo el API Gateway unifica el acceso a todos los microservicios de ChaskiPC y aplica balanceo de carga automático sin exponer la topología interna.

2. ¿Qué punto de la clase te resultó más confuso o te dejó con dudas?

La sincronización entre el refresco de rutas del Gateway y el tiempo de arrendamiento (lease renewal) de Eureka.

3. ¿Tienes alguna pregunta que te gustaría que sea respondida la siguiente clase?

¿Cómo se implementan filtros globales de autenticación JWT en Spring Cloud Gateway para validar tokens antes de reenviar la petición a los microservicios de negocio?

4. Sobre tu nivel de comprensión de la clase de hoy, marca una opción:

☒ ¡Entendido! - Lo domino y podría explicarlo.

5. ¿Cómo puedo ayudarte a comprender mejor el tema?

Incluyendo ejemplos prácticos de Rate Limiting y Circuit Breaker (Resilience4j) en el Gateway.

6. Pensando en tu participación y esfuerzo en la clase de hoy, ¿cómo te autoevaluarías?

☒ Muy Comprometido/a: Me esforcé al máximo.

7. Mi satisfacción con la clase fue:

10 / 10